

プロジェクト演習 テーマD

第5回

担当：CS学部 教授 伏見卓恭
連絡先：fushimity@edu.teu.ac.jp

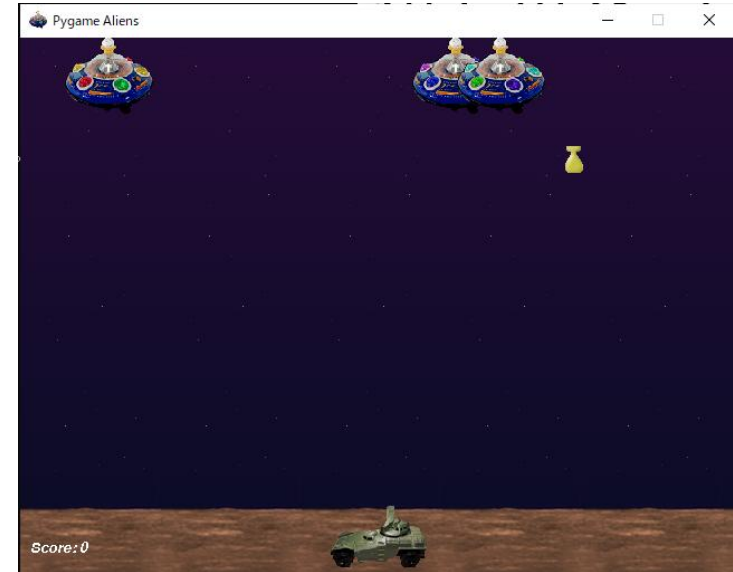
授業の流れ

- 第1回：実験環境の構築 / Gitの復習 / Pygameの基礎
- 第2回：関数を用いたゲーム開発の基礎 / コード規約とコードレビュー
- 第3回：オブジェクト指向によるゲーム開発 / ブランチとマージによる開発
- 第4回：Pygameらしいゲーム開発 / フォークとプルリクによる共同開発
- 第5回：共同開発演習（ベース部分＋個別機能実装）
- 第6回：共同開発演習（統合と調整）
- 第7回：共同開発演習（成果発表）

サンプルゲームで遊んでみよう

- pygameモジュール内のexamplesにあるサンプルゲームを実行する

__pycache__	2022/03/29 17:40	ファイル フォルダー	
data	2022/03/29 17:40	ファイル フォルダー	
__init__.py	2022/03/29 17:40	PY ファイル	0 KB
aacircle.py	2022/03/29 17:40	PY ファイル	2 KB
aliens.py	2022/03/29 17:40	PY ファイル	12 KB
arraydemo.py	2022/03/29 17:40	PY ファイル	4 KB
audiocapture.py	2022/03/29 17:40	PY ファイル	2 KB
blend_fill.py	2022/03/29 17:40	PY ファイル	4 KB
blit_blends.py	2022/03/29 17:40	PY ファイル	7 KB
camera.py	2022/03/29 17:40	PY ファイル	3 KB
chimp.py	2022/03/29 17:40	PY ファイル	6 KB
cursors.py	2022/03/29 17:40	PY ファイル	3 KB
dropevent.py	2022/03/29 17:40	PY ファイル	3 KB
eventlist.py	2022/03/29 17:40	PY ファイル	6 KB
font_viewer.py	2022/03/29 17:40	PY ファイル	10 KB



VScodeのターミナルにて, 以下を実行する

```
PS C:¥Users¥Admin¥Desktop¥ProjExD> python -m pygame.examples.aliens
```

↑

「python -m モジュール名」でモジュールのプログラムを実行できる

【参考】pygameに関する情報の確認

- VScodeのターミナルで, pipコマンドにより確認する

```
PS C:\Users\Admin\Desktop\ProjExD> pip show pygame
Name: pygame
Version: 2.5.2
Summary: Python Game Development
Home-page: https://www.pygame.org
Author: A community project.
Author-email: pygame@pygame.org
License: LGPL
Location: c:\users\admin\anaconda3\envs\projexd\lib\site-packages
Requires:
Required-by:
```

←大文字のフォルダ名も
小文字で表示されている

```
Author-email: pygame@pygame.org
License: LGPL
Location: c:\users\admin\anaconda3\envs\projexd\lib\site-packages
```

フォルダーを新しいウィンドウで開く (Ctrl + クリック)

【参考】効果音 / BGMの出し方

参考 : ex1/sound_sample.py

- 最初にmixerを初期化する : `pg.mixer.init()`
- 効果音
 - 音源ファイルを読み込む (クラス変数など) :
`snd = pg.mixer.Sound(ファイルパス)`
 - 再生する : `snd.play(maxtime=最大ミリ秒)` ← 引数なしで音源ファイルの長さ
- BGM
 - 音源ファイルを読み込む (ゲームループ前など) :
`pg.mixer.music.load(ファイルパス)`
 - 再生する : `pg.mixer.music.play(loops=ループ回数)` ← `loops=-1` で無限ループ
 - 停止する : `pg.mixer.music.stop()`

今後の流れ

【第5回】

- ～15時：実装するゲームの相談
 - 共通基本機能と追加機能（人数分）に分割し，分担を決める
 - コード規約や変数名，クラス設計などの相談
- ～次回授業開始：共通基本機能＋担当追加機能の個別実装
 - メンバーで相談しながら実装してもOK
- ～次回授業開始：提出物の作成

【第6回】

- ～17時：担当機能の完成 → コードのマージ → ゲーム完成＋README完成
- ～18時：提出物の作成

【第7回】

- ～14時：発表準備（パワポ不要）
- ～18限：成果発表会（デモンストレーション，コードの説明）

次回までの宿題

- 次回授業開始までに、自分の担当機能の実装を完了させること
- 提出物をMoodleにアップロードすること
- 提出物チェックは、次回の3限に行う
- 本日提出できる場合は、本日チェックを受けること

注意事項

- 条件：
 - pygameを用いること
 - 「import os」→
「os.chdir(os.path.dirname(os.path.abspath(__file__)))」を書くこと
- 進め方
 - あとでマージすることを念頭において、機能分割、役割分担、コード規約（変数名の付け方など）、クラス設計など念入りに検討・相談して進めること
- ゲームの内容
 - メンバーのプログラミング能力を鑑みて、ゲーム内容を決定
 - ネット上にある記事などを参考にしてもよいが、丸パクリはダメ
 - 差分が小さければ、加点も小さい
 - 複数サイトからコピーしてコードに一貫性がなくなれば、減点になる

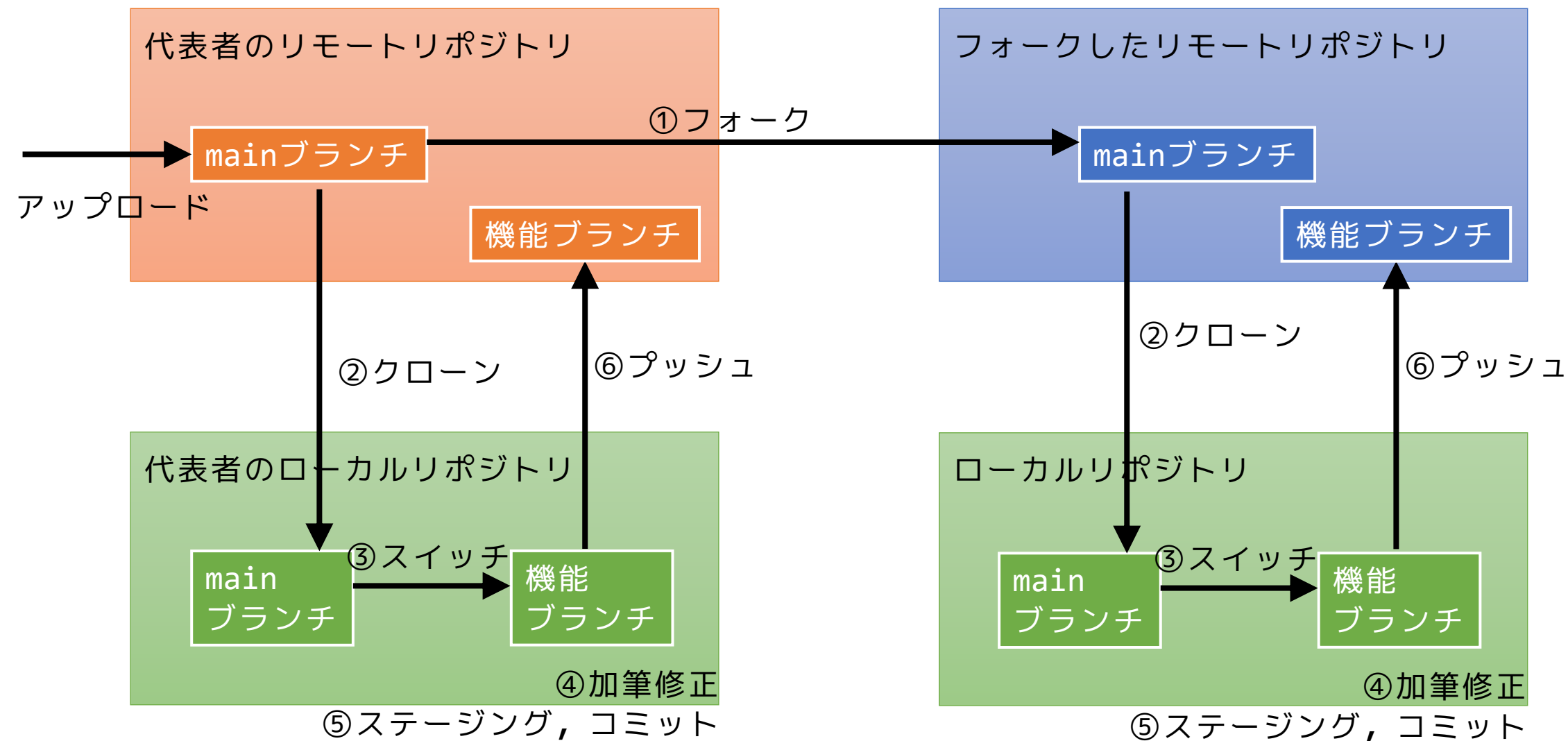
基本機能（ベース部分）と追加機能（個人実装部分）の分け方

- 基本機能（ベース部分）の例
 - 授業で配布したコード（練習や演習で追記した部分も含む）
 - インターネット上のコード
 - 生成AIによるコード
 - オリジナルの場合は、「背景画像の表示のみ」などが書かれたコード
- 追加機能（個人実装部分）
 - 上記の基本機能に、各自で**然るべき量のコードと機能**を追加すること
 - 第4回で個人実装した機能と同等かそれ以上
 - 本日3コマと来週までの時間があるので、2,3行であるはずがない

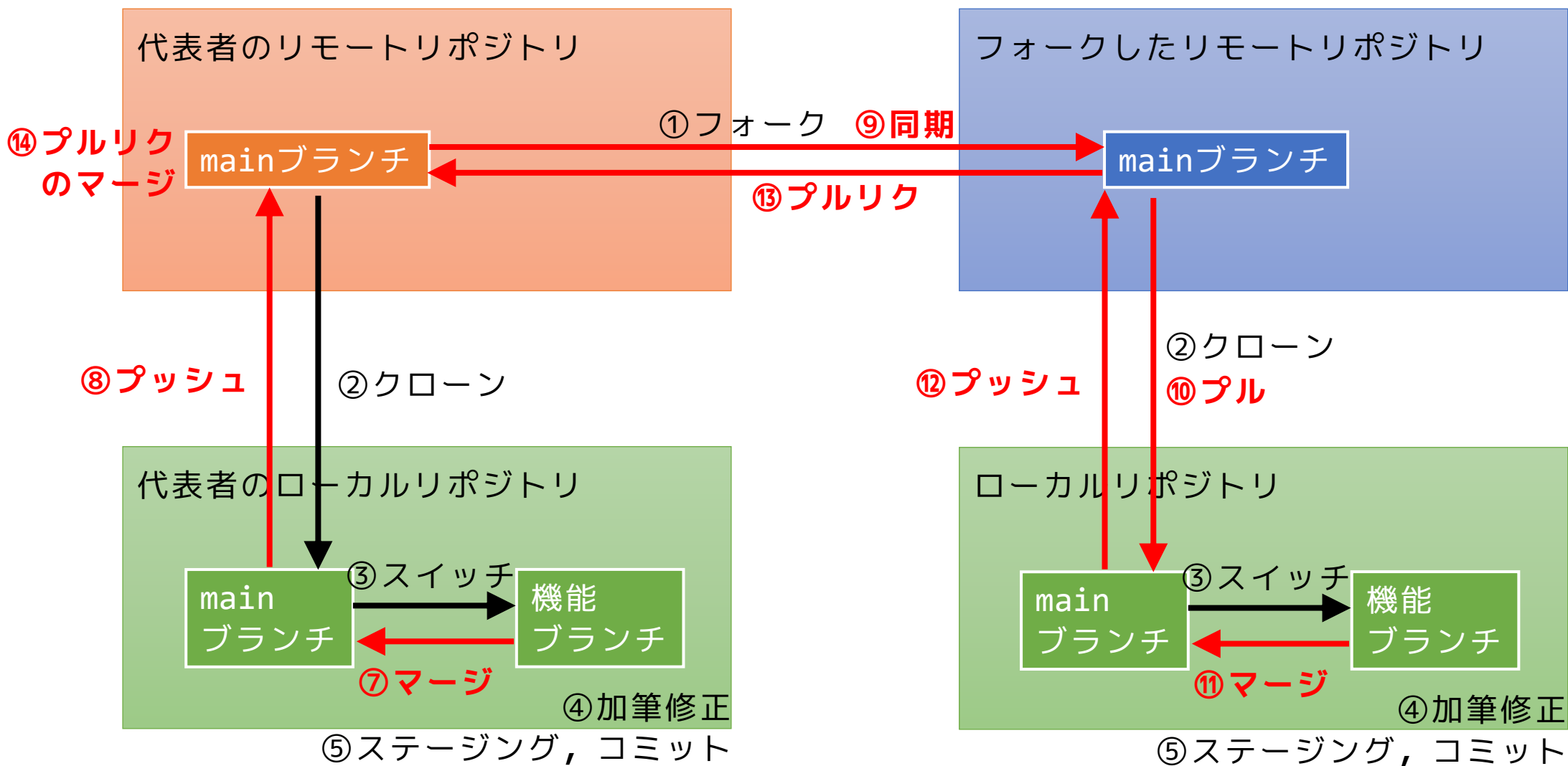
作業の流れ（全体像）

1. 代表者を決める ←次回，代表者加点あり
2. 代表者は，公開リポジトリを作成し，README.mdをアップロードする
 - 公開リポジトリ名：ProjExD_Group0X
3. 代表者は，上記リポジトリをex5という名前でクローンする
4. 代表者は，メンバーと相談して基本機能（ベース），README.mdを作成する
 - 必ず「`os.chdir(os.path.dirname(os.path.abspath(__file__)))`」を書くこと
5. 代表者は，コード一式とREADME.mdをステ，コミ，プッシュする
6. その他メンバーは，上記の共用リポジトリをフォークする
7. その他メンバーは，フォークしたリモートリポジトリをローカルにクローンする
8. ローカルに機能ブランチを作り，担当機能を実装する+READMEを追記する
 - ブランチ名：C0A25XXX/機能名
9. 完成した機能ブランチを自身のリモートリポジトリにプッシュする ←本日の提出物
 - 機能ブランチにて：`git push origin C0A25XXX/機能名`

今回の作業の流れ（イメージ図）



【参考】 次回の作業の流れ（イメージ図）



README.mdの例

①最初に，相談した内容を代表者がメモをする

②個人実装時に各自で追記，修正する

※共通基本機能と自分が担当した追加機能がわかるようにすること

丸焼き豪華豚 ← ゲームのタイトル（仮）

実行環境の必要条件

* python >= 3.10

* pygame >= 2.1

ゲームの概要

* 主人公キャラクター豪華豚（ゴウカトン）をマウス操作により丸焼きにするゲームで，...

* 参考URL：[サイトタイトル](https://www.hoge.com/)

ゲームの実装

共通基本機能

* 背景画像と主人公キャラクターの描画

分担追加機能

* 丸焼きエフェクト（担当：ふしみ）：バーナーにより豪華豚を丸焼きにするエフェクトに関するクラス

* キッチンタイマー機能（担当：ふしみ）：制限時間以内に調理が完了しなかった場合に，豪華豚が脱走する機能

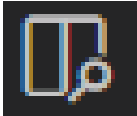
* 調理機能（担当：ふしみ）：調理器具をキー押下により選択し，豪華豚を調理する機能

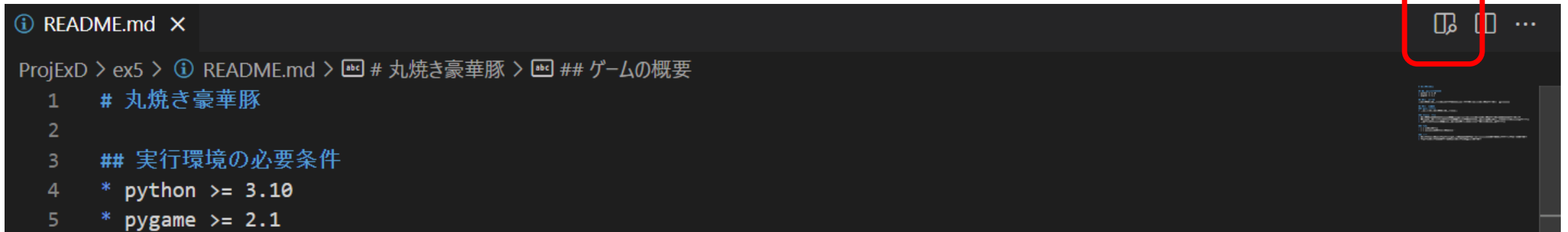
メモ ← グループメンバーへの連絡など

* クラス内の変数は，すべて，「get_変数名」という名前のメソッドを介してアクセスするように設計してある

* すべてのクラスに関係する関数は，クラスの外で定義してある

VScodeでプレビューしながら編集

- いつも通りREADME.mdを開く
- 右上のボタン , または, Ctrl+K Vでプレビューを横に表示する



```
ProjExD > ex5 > README.md > # 丸焼き豪華豚 > ## ゲームの概要
1 # 丸焼き豪華豚
2
3 ## 実行環境の必要条件
4 * python >= 3.10
5 * pygame >= 2.1
```



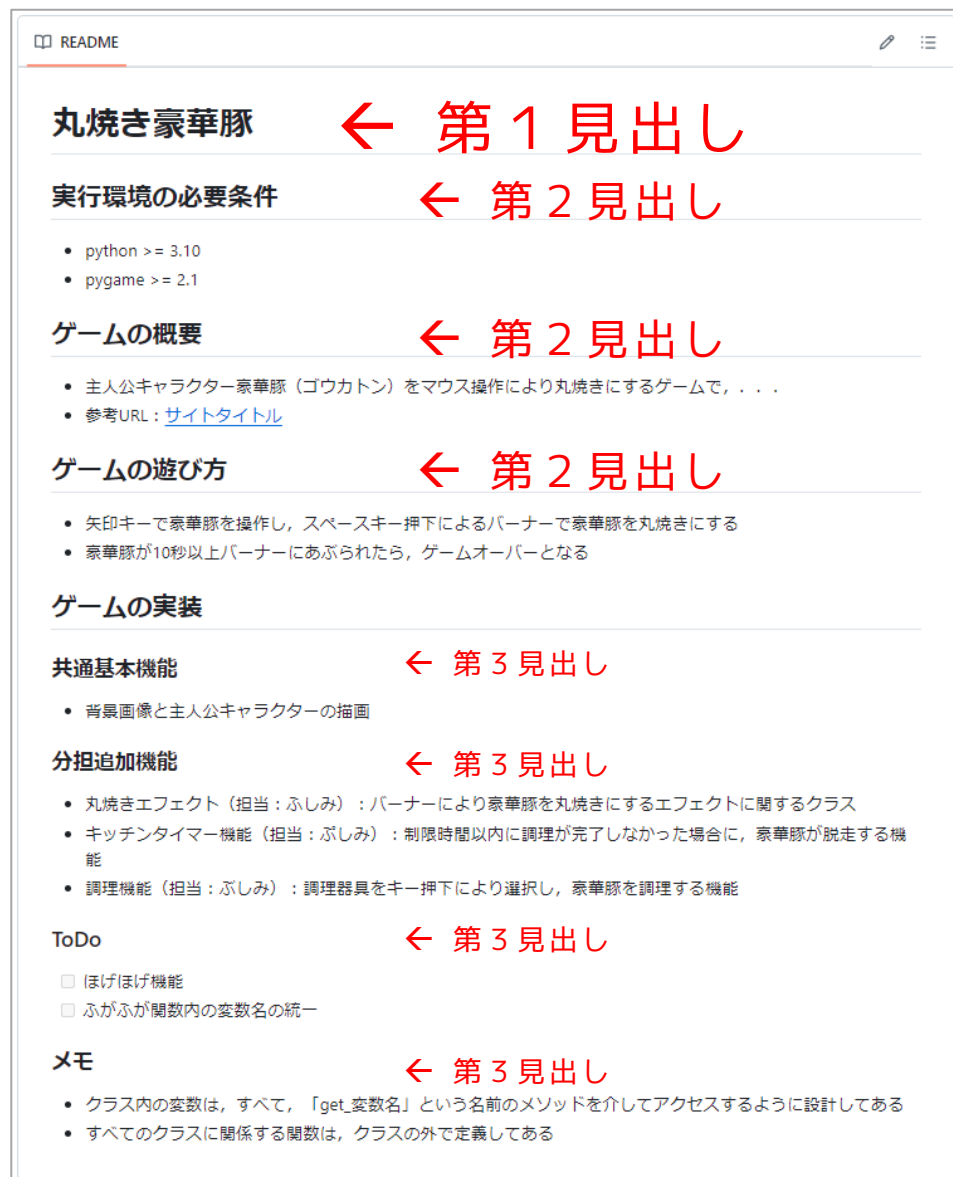
```
ProjExD > ex5 > README.md > # 丸焼き豪華豚 > ## ゲームの概要
1 # 丸焼き豪華豚
2
3 ## 実行環境の必要条件
4 * python >= 3.10
5 * pygame >= 2.1
6
7 ## ゲームの概要
8 主人公キャラクター豪華豚（ゴウカトン）をマウス操作により丸焼きにするゲー
```

丸焼き豪華豚

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1

README.mdをプレビュー表示すると



マークダウン記法

書き方	範例
# 見出し	見出し
- 順序なしリスト	• 順序なしリスト
1. 順序付きリスト	1. 順序付きリスト
- [] チェックリスト	☐ チェックリスト
> 引用	引用
太字	太字
斜体	斜体
~~取消線~~	取消線

[link text](https:// "title")	リンク
![image alt](https:// "title")	画像
`コードブロック`	コードブロック
````javascript var i = 0; ````	<pre>1   var i = 0;</pre>
:smile:	

参考URL : <https://qiita.com/tbpgr/items/989c6badefff69377da7>

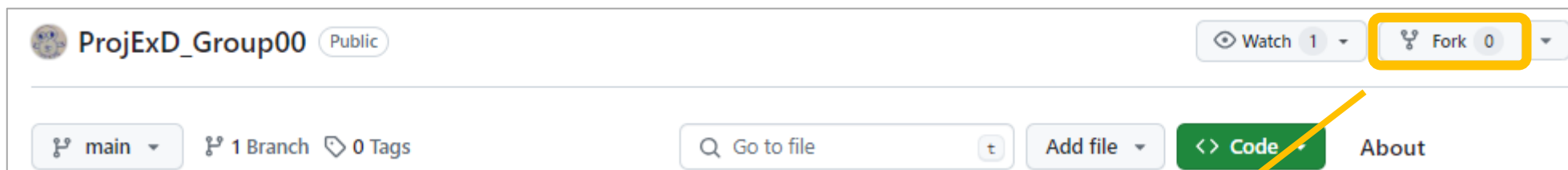
参考URL : <https://qiita.com/jkjoluvjlajelljicvjzobjieoaid/items/01cd7ff819bc2e69b652>



# フォークのやりかた

(全) : 全員  
(代) : 代表者  
(他) : 代表者以外

- (他) 代表者のリポジトリを自身のリモートリポジトリとしてフォークする
  - 代表者のリポジトリで, 「Fork」 → 「Create fork」



**Create a new fork**

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner *  / Repository name *

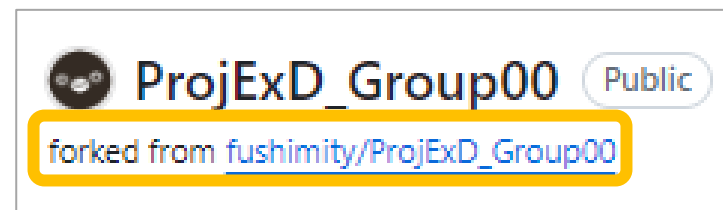
☒ ProjExD_Group00 is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)

☒ Copy the **main** branch only  
Contribute back to fushimity/ProjExD_Group00 by adding your own branch. [Learn more.](#)

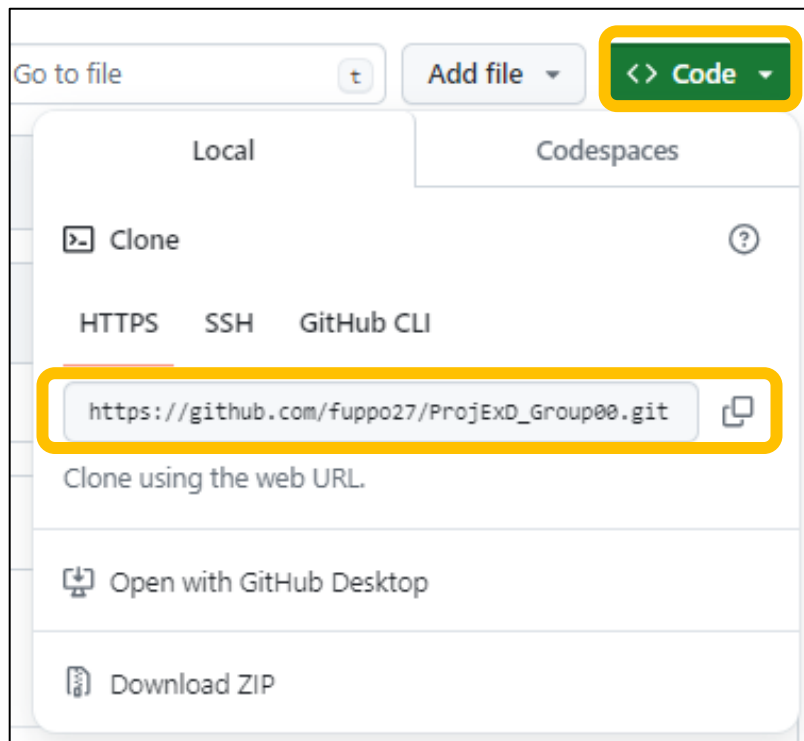
① You are creating a fork in your personal account.



↑ 自身のリポジトリ名の下に  
「forked from 代表者リポ名」  
が表示される

# クローンのやりかた

- (他) ProjExDフォルダにて, フォークしたリポジトリをクローンする :  
`git clone フォークしたリポジトリのURL ex5`



※フォークしたリポジトリ  
= 自分のアカウントにあるリポジトリ

# 代表者がmainブランチを更新したら

- 代表者が共通基本機能やREADMEに変更を加えたり、ファイルを追加したりなど、mainブランチを更新したら、  
その他メンバーは自身のローカルリポジトリのmainブランチを  
**その都度同期**させ、最新の状態を保つようにする
  - (代)メンバーに更新したことを伝える
  - (他)リモートリポジトリを同期する: Sync fork
  - (他)現在実装中の内容をステコミする: **git add 略 / git commit 略**
  - (他)mainブランチにスイッチ: **git switch main**
  - (他)同期したリモートリポジトリをプルする: **git pull origin main**
    - (他)マージコミットが必要なら、VScodeでコミットコメントを入力する
    - (他)コンフリクトが起きたら、VScodeで修正する(現時点では起きないはず)

# コンフリクトの自動解消方法

- コンフリクト：同じ場所に異なる編集が加わっている状態
- 解消方法：「**どちらか一方だけが正しいというわけではない**」ので、それぞれの編集意図を組み、相談しながら慎重にコードを修正する
- 必要に応じて、加筆しても構わない
- 【手順】
  1. マージエディタでコードを修正する
  2. 保存する
  3. コードが実行できることを確認する
  4. ステージング, コミットする

マージコミットと呼ばれる ↑

```
6 Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
7 <<<<<< HEAD (Current Change)
8 idList := [...]map[string]string{
9 {"ID": "1", "AID": "c1", "BID": "b1"},
10 {"ID": "2", "AID": "c2", "BID": "b2"},
11 {"ID": "3", "AID": "c3", "BID": "b3"},
12 }
13
14 mapList := []map[string]string{}
15 for _, i := range idList {
16 data := map[string]string{
17 "id": i["ID"],
18 "aaa_id": i["AID"],
19 "bbb_id": i["BID"],
20 "memo": "移動済み",
21 }
22 mapList = append(mapList, data)
23 }
24 fmt.Print(mapList)
25 }
26
27 =====
28 idList := [...]map[string]string{
29 {"ID": "1", "AID": "a1", "BID": "b1"},
30 {"ID": "2", "AID": "a2", "BID": "b2"},
31 {"ID": "3", "AID": "a3", "BID": "b3"},
32 }
```

どちらか一方選択するだけでなく、場合によっては両方を残す

# チェック項目：個人成果物(15点満点)

0: 不可  
1: 可  
2: 優

1. READMEの説明が適切か [0 -- 2]

2. 基本機能 (ベース部分) と比較して十分な量のコードか [0 -- 4]

3. どこかのサイトのパクリ or AI生成コードでないか [0 or 1]

4. コード内コメントが必要十分か [0 -- 2]

5. 型ヒント, 関数アノテーション, docstringなどは十分か [0 -- 2]

追加機能部分

6. コード規約が (ある程度) 守られているか [0 or 2]

7. 関数, クラスを定義・使用した可読性の高いコードか [0 -- 2]

コード全体

8. 提出物不備 (ファイル名, クリックابل, 内容物) は1点ずつ減点

# 提出物

学籍番号は, 半角・大文字で

- ファイル名 : C0A25XXX_kadai5.pdf
- 内容 : 以下の順番でPDFを結合して提出すること

- README.md

[https://github.com/c0a25xxx/ProjExD_Group00/blob/branch/README.md](https://github.com/c0a25xxx/ProjExD_Group00/blob/branch/README.md)

- コード

[https://github.com/c0a25xxx/ProjExD_Group00/blob/branch/?????.py](https://github.com/c0a25xxx/ProjExD_Group00/blob/branch/?????.py)

- 基本機能（ベース部分）との差分

[https://github.com/c0a25xxx/ProjExD_Group00/compare/main...branch](https://github.com/c0a25xxx/ProjExD_Group00/compare/main...branch)

自分のアカウント名

グループ番号

自分の機能ブランチ名

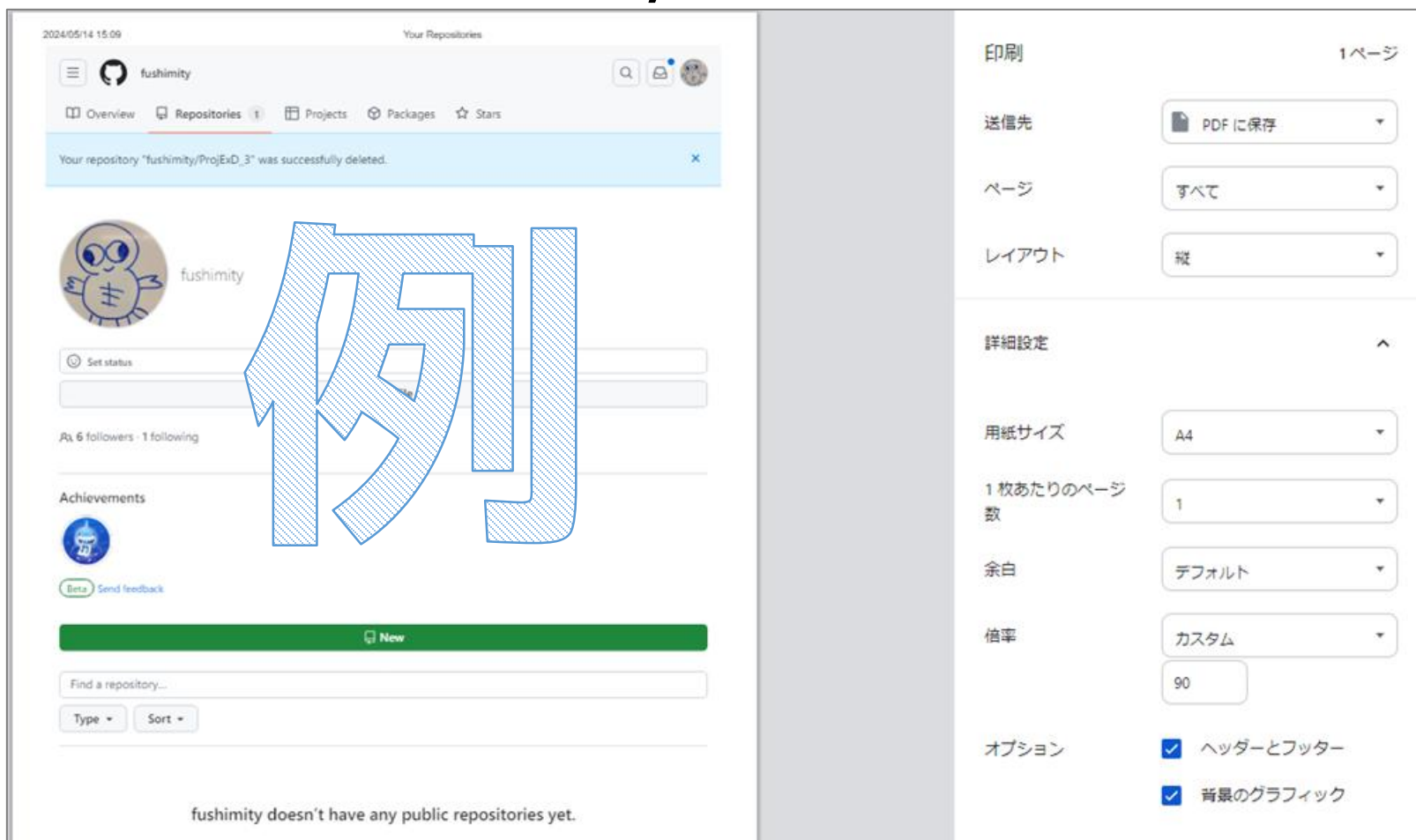
# 提出物の作り方

※スクショ画像は認めません。  
以下の手順に従ってPDFを作成し、提出すること

1. ChromeでPDFとして保存する（次ページを参照）
2. 以下のURLから各PDFをマージする  
[https://www.ilovepdf.com/ja/merge_pdf](https://www.ilovepdf.com/ja/merge_pdf)
3. ファイル名を「C0A25XXX_kadai5.pdf」として保存する

# ChromeでPDFとして保存する方法

1. 該当ページを表示させた状態で「Ctrl+P」
2. 以下のように設定し、「保存」をクリックする



Microsoft Print to PDFはダメ

←送信先：PDFに保存

←ページ：すべて

←レイアウト：縦

←用紙サイズ：A4

←余白：デフォルト

←倍率：90

←両方チェック



# チェックの手順

※基本的に、TASAチェックは1度だけですので、  
慎重に提出物PDFを作ること。どうしてもの場合は要相談。

1. 受講生：提出物（pdf）を作成する
2. 受講生：担当TASAに提出物と成果物（ゲーム）を見せに行く
3. TASA：提出物とゲームのデモを確認し、点数を確定する
4. 受講生：提出物をMoodleにアップロードして、帰る
5. FSM：近日中に課題と点数を確認し、Moodleに登録する

- 時間内にチェックが終わらなそうな場合は、  
提出物をMoodleに提出してから帰る  
（次回までor次回の3限にチェックされる）

← 授業後に提出した場合  
時間外提出扱いになり  
割引いて採点するので  
できるだけ提出して、  
チェックを受けること